COMP 018: Database Administration - Activity 5 Progress Report
Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course : COMP 018 - Database Administration
Evaluating Group : Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository : https://github.com/Nycri/brewpos
Target Project Monitored : OJT Management System (Project-OJT-RMA-System)
Target Project Group : Group 3 (Evaluated under Tracker Key: Group 06 / Group 03): Simon, De Dios, Ramos, Dela Rosa, Sietereales, Geraldez
Target Project Repository : https://github.com/swvenz/Project-OJT-RMA-System (Branches: main, ojt-system-untested.demo)
Primary Database : ojt_management (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database : ojt_management_restore_test

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection
Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 9.0 megabytes across twelve base tables and twelve sequences within the public schema. Table storage analysis showed that user profile tables account for 64 kilobytes, student records account for 56 kilobytes, and application, requirement, and company tables account for 48 kilobytes each. Placement, attendance, and supervisor tables occupy 40 kilobytes each, while evaluations and audit logs occupy 32 kilobytes each.

2.2 Connection, Session & Activity Monitoring
Database statistics recorded one active connection with 116 committed transactions and 10 rolled back transactions. Zero lock conflicts and zero deadlocks were recorded across sessions, confirming stable transaction processing. The rolled back transaction count reflected expected error conditions intentionally triggered during negative security verification testing. System catalog queries confirmed that active roles and memberships accurately reflect access control rules.

2.3 Table Statistics & Bloat Inspection
Pre-maintenance table statistics revealed dead tuple accumulation within the application table resulting from update and deletion operations executed during Activity 4 verification testing. Remaining operational tables exhibited zero dead tuples. The presence of obsolete row versions confirmed the necessity of running routine vacuuming and statistics updating operations.

2.4 Log Review & Issue Identification

Review of database logs verified that unauthorized access attempts during Activity 4 security testing were recorded by the engine. Logged events included permission denials on deletion commands attempted on student, attendance, and evaluation tables, denied query attempts on credential tables, blocked database creation requests, and denied cross-database connection attempts. Maintenance boundary enforcement logs confirmed that runtime accounts cannot initiate vacuuming operations. Server background processes completed cleanly without system crashes or storage warnings.

2.5 Routine Maintenance Operations

Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. Initial testing confirmed that non-owner application runtime accounts are denied execution of maintenance commands, proving that maintenance tasks require administrative credentials.

Full database statistics updates executed cleanly under administrative credentials. Standard vacuuming on the application table scanned the relation, removing obsolete row versions and updating frozen transaction counts. Vacuum processing on attendance and evaluation tables verified table health and refreshed statistics without locking issues.

Full table rewriting vacuum operations were omitted because such commands acquire exclusive locks that block concurrent application read and write operations. Because relations reside within single disk pages, standard non-blocking vacuuming reclaimed dead tuples effectively without disrupting application availability.

2.6 Query Performance Analysis

Performance analysis was conducted on a student placement lookup query calculating verified attendance hours. The query executed in 0.140 milliseconds with complete buffer cache hit efficiency. The query planner executed nested loop joins across placement, student, and company tables, processing subqueries for aggregate hours directly in memory without disk I/O.

Analysis of a requirement compliance query joining student submissions with requirements catalogs yielded an execution time of 0.126 milliseconds with zero physical disk reads. The query engine built in-memory hash tables across join relations, executing chained hash joins within dedicated memory structures without disk spills.

2.7 Prioritized Recommendations & Technical Justification

The current index configuration is appropriate for operational requirements, as active queries execute under 0.15 milliseconds with complete buffer cache hits. Immediate index additions are unnecessary and would introduce write overhead during record updates. If attendance log volumes increase significantly in production, adding a composite partial index on unverified attendance records will optimize supervisor review queries. Deploying a scheduled weekly background maintenance task will maintain page efficiency during off-peak hours.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	Value
Project	Group 3 - OJT Management System (Project-OJT-RMA-System) [Tracker Key: Group 06 / Group 03]
Database inspected	ojt_management (PostgreSQL 18.6)
Account and role used	postgres (DBA Maintenance), ojt_app (Runtime), ojt_report_user (Audit)
Health check result	Healthy (0 conflicts, 0 deadlocks, 9.0 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on ojt_applications, attendance, evaluations via postgres
Query analyzed	Student placement progress & verified hours (Nested Loop, 0.140 ms, 7 buffer hits) & Requirements compliance review (Hash Joins, 0.126 ms)
Recommendation priority	No action required (Current queries execute in < 0.15 ms with 100% buffer cache hits)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evidence-act5
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity5_Progress_Report_Group3.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed

and validated against PostgreSQL 18.6 within the local database environment.